Friedrich-Alexander-Universität
Technische Fakultät





Computer Architecture Practical Exercise

2 Loop Unrolling

Kenan Gündogan[1] **Philipp Gündisch**[1]

[1]Friedrich-Alexander Universität Erlangen-Nürnberg, Chair of Computer Science 3 (Computer Architecture)

November 14, 2023

Motivation

Compiler Optimization

- Compilers implicitly perform code optimization techniques
- With `icc -O3 -xHost` aggressive optimization for the specific host machine is requested
- Two very effective optimization techniques are called:
  - **Loop Unrolling** (this week)
  - **Vectorization** (next week)
- We will implement these techniques manually to understand their implications on the performance

Disable Compiler Optimizations

Before implementing these optimization techniques we need to disable the implicit compiler optimizations This can be done with compiler directives (`#pragma`)

- Disable **Loop Unrolling** with `#pragma nounroll`
- Disable **Vectorization** by
  - the use of `#pragma novector`
  - and removing `-xHost` from compiler flags

Example applied to `vec_sum()`:

```
#include "vec_sum.h"

float vec_sum(float *array, uint32_t length) {
        float sum = 0.0f;
        #pragma nounroll
        #pragma novector
        for(uint32_t i = 0 ; i < length ; i++) {
                sum += array[i];
        }
        return sum;
}
```

Loop Unrolling

Introduction

Loops can be manually unrolled as displayed in the following example.

```
#include "vec_sum.h"

float vec_sum(float *array, uint32_t length) {
        float sum[2] = { 0.0f, 0,0f };
        uint32_t remainder = length % 2;

        #pragma nounroll
        #pragma novector
        for(uint32_t i = 0 ; i < length-remainder ; i+=2) {
                sum[0] += array[i];
                sum[1] += array[i+1];
        }

        sum[0] += sum[1];
        if(remainder)
                sum[0] += array[length-1];
        return sum[0];
}
```

Loop Unrolling

Assembler Code

Loop Unrolling reduces the control flow overhead of loops.

Assembler code resembling the for loop `vec_sum` without loop unrolling.

```
..B1.5:
        addss   (%rdi,%rax,4), %xmm0
        incl    %eax
        cmpl    %esi, %eax
        jb      ..B1.5
```

Same code with twofold loop unrolling.

```
        shrl    $1, %eax
..B1.7:
        lea     (%rdx,4), %r8d
        lea     1(%rdx,4), %r9d
        incl    %edx                        # Note: it is still +1
        addss   (%rdi,%r8,4), %xmm0
        addss   (%rdi,%r9,4), %xmm1
        cmpl    %eax, %edx
        jb      ..B1.7
```

Note: the assembler codes were produced with:

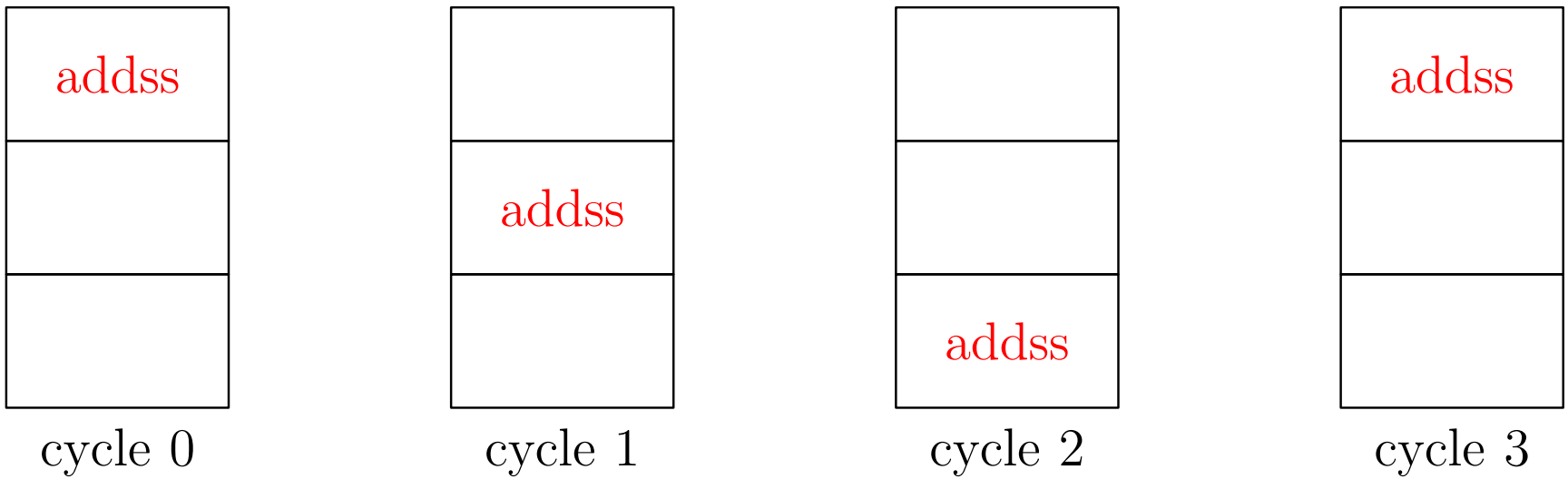
`icc -I ./include/ -S -O3 vec_sum.c`

Loop Unrolling

No Unrolling - Effects on Pipelined Units

```
#pragma novector
#pragma nounroll
for(i = 0; i < n; i += 1) {
        sum += A[i];
}
```

```
..B1.5:
        addss   (%rdi,%rax,4), %xmm0
        incl    %eax
        cmpl    %esi, %eax
        jb      ..B1.5
```

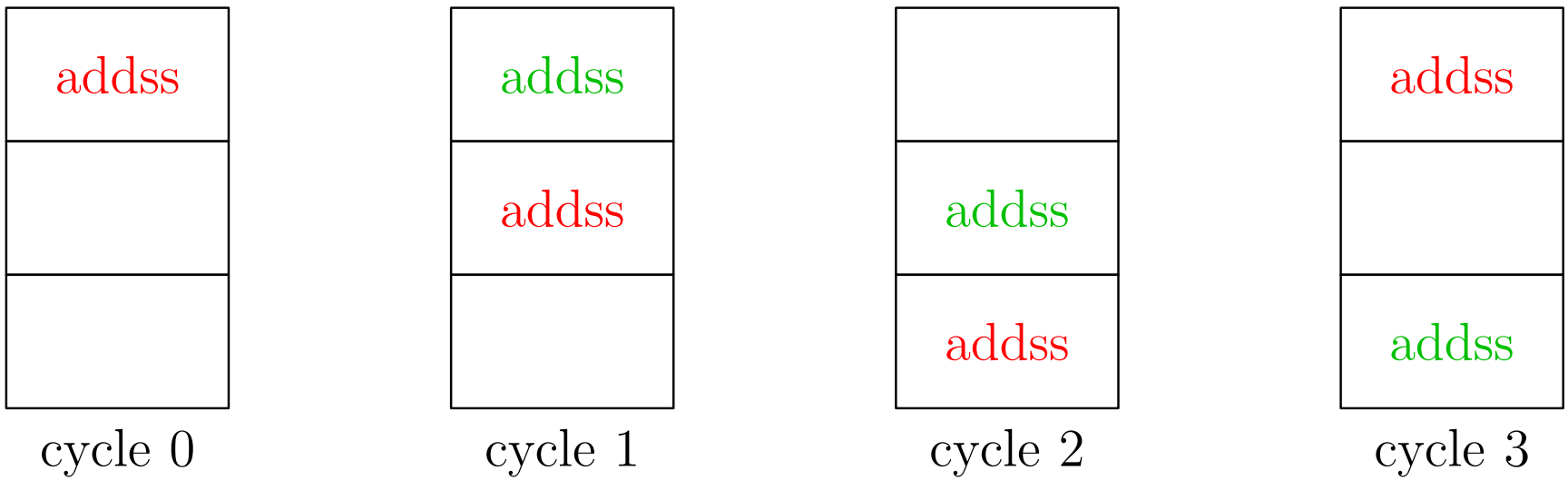


Loop Unrolling

2-fold Unrolling - Effects on Pipelined Units

```
#pragma novector
#pragma nounroll
for(i = 0; i < n; i += 2) {
        sum0 += A[i];
        sum1 += A[i+1];
}
```

```
..B1.7:
        lea     (%rdx,4), %r8d
        lea     1(%rdx,4), %r9d
        incl    %edx
        addss   (%rdi,%r8,4), %xmm0
        addss   (%rdi,%r9,4), %xmm1
        cmpl    %eax, %edx
        jb      ..B1.7
```

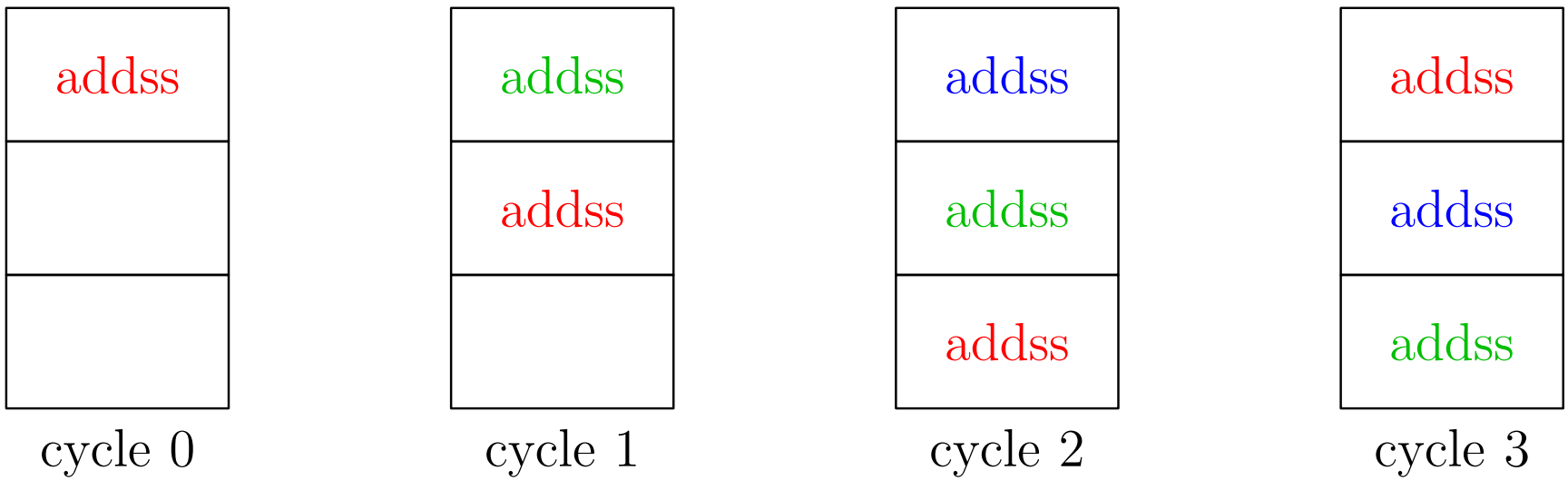


Loop Unrolling

3-fold Unrolling - Effects on Pipelined Units

```
#pragma novector
#pragma nounroll
for(i = 0; i < n; i += 3) {
        sum0 += A[i];
        sum1 += A[i+1];
        sum2 += A[i+2];
}
```

```
..B1.7:
        incl    %ecx
        lea     1(%rax), %r10d
        lea     2(%rax), %r11d
        addss   (%r8,%r10,4), %xmm3
        addss   (%r8,%rax,4), %xmm0
        addss   (%r8,%r11,4), %xmm2
        addl    $3, %eax
        cmpl    %edx, %ecx
        jb      ..B1.7
```



Task 2.1: VecSum Loop Unrolling

Implementation

- Adapt `vec_sum.c` to suppress unrolling and vectorization.
- Adapt the loop of `vec_sum()` to support benchmarking of:
  - No unrolling
  - Twofold unrolling
  - Threefold unrolling
  - Fourfold unrolling
  - Eightfold unrolling

  Hint: Implement solutions in separate files or use `#ifdef`.
- Benchmark each implementation for 1KiB - 128MiB (1 second runtime)

Task 2.2: Jacobi Loop Unrolling

Implementation

- Adapt `jacobi.c` to suppress unrolling and vectorization.
- Adapt the innermost loop of `jacobi()` to support benchmarking of:
  - No unrolling
  - Twofold unrolling
  - Fourfold unrolling

  Hint: Implement solutions in separate files or use `#ifdef`.
- Benchmark each implementation for 1KiB - 128MiB (1 second runtime)
- Why does jacobi unrolling not show the same performance gains as vec_sum?

Task 2.3: Plot Benchmark Results

Visualization

Visualize the benchmark results with the tool of your choice:

- Plot Jacobi and VecSum into different plots
- Draw a line chart with each line resembling an unrolling variant
- Choose the memory consumption as the X-axis
- Choose the performance metrics for the Y-axis (AdditionsPerSecond, MUp/s)

Task Overview

- E 2.1: VecSum Loop Unrolling
  - No unrolling
  - Twofold unrolling
  - Threefold unrolling
  - Fourfold unrolling
  - Eightfold unrolling
- E 2.2: Jacobi Inner Loop Unrolling
  - No unrolling
  - Twofold unrolling
  - Fourfold unrolling
- E 2.3: Benchmark Result Plotting
  - One VecSum plot comparing the unrolling variants
  - One Jacobi plot comparing the unrolling variants

Appendix: `define`

Best Practice

Defines can be looked at as compile time variables. They are evaluated by the preprocessor of the compiler. Since the compiler knows their values, optimizations can be applied during the compilation process. This is a huge advantage over standard C variables.

```
#ifndef NUMBER                      // Check if it was previously defined
        #define NUMBER (3+4)        // Hint: Always use braces
#endif

// Use of conditionals
#if NUMBER == 7
        printf("NUMBER␣IS␣7\n");
#elif NUMBER > 7
        printf("NUMBER␣IS␣GREATER␣THAN␣7\n");
#else
        printf("NUMBER␣IS␣LESS␣THAN␣7\n");
#endif
```

Defines can be passed to the compiler with `-D`:

```
icc -DNUMBER=7 ...
```

Appendix: Checklist

Performance Optimization

During the timeline of this class new bullet points will be added.

- Compiling
  - Choice of the compiler (`icc`)
  - Compiler flag to optimize aggressively (e.g. `-O3`)
  - Compiler flag to adapt for specific hardware (e.g. `-xHost`)
- Programming Techniques (if applicable)
  - Use `#define` and `const` instead of variables
  - Data type aware programming
  - Use aligned memory (e.g. with `_mm_malloc()` or `posix_memalign()`)
  - Consecutive address iteration
- Measurement
  - Reasonable benchmark time
  - Reasonable benchmark workload
  - Reduce interference factors to a minimum